

Engenharia de Software – PUC Campinas

Algoritmos e Linguagens de Programação

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Funções

Exercícios

○ ANTES DE COMEÇAR OS EXERCÍCIOS:

- Neste momento do curso, o foco é o desenvolvimento de funções — trechos de código bem definidos, reutilizáveis e independentes.
- Cada exercício desta lista descreve um problema que deve ser resolvido com uma função bem projetada.
- Para isso, siga estas orientações:
 - Comece escrevendo a função. Não comece pelo main. O main serve apenas para testar a função — e não o contrário.
 - Pense na função como uma "caixa-preta": ela deve receber somente os parâmetros estritamente necessários para executar sua tarefa e retornar um resultado (se for o caso), sem depender de variáveis externas ou interações com o usuário.
 - A função não deve imprimir nada (a menos que o enunciado peça). O teste da função com printf, scanf ou main() é feito à parte.
 - Seja direto. A função deve ser concisa, clara e focada. Evite colocar dentro dela partes do código que pertencem à lógica do programa principal.
 - Use o main() apenas como espaço de testes: você pode escrever diversos printf ali para verificar se a função está funcionando como deveria.
- Ao pensar cuidadosamente sobre os parâmetros e o retorno da função, você estará praticando um dos aspectos mais importantes da programação: abstração.

Exercícios

OBS.: 1. Cada questão poderá utilizar funções implementadas nos exercícios que a antecederam!!!

2. Façam todas as funções no mesmo arquivo pois serão todas necessárias na questão 16.

- 1) Implemente uma função que recebe um valor inteiro e verifica se o valor é par. A função deve retornar um booleano e o resultado deverá ser impresso no programa principal.
- 2) Implemente duas funções:
 - Uma que lê do teclado um número real e o retorna.
 - Outra que a partir do número real lido retorna a parte fracionária do mesmo.
- 3) Implemente uma função que dado um numero inteiro positivo ela retorne o seu fatorial.
- 4) Implemente uma função que recebe dois valores inteiros: a e b, e escreve os números pares no intervalo de "a" a "b", inclusive ambos se estes forem pares. O programa deve funcionar mesmo que a seja maior que b.

Exercícios

- 5) Implemente uma função que receba uma string (array de char) e retorne o seu tamanho (quantidade de caracteres antes do caractere nulo '\0').
- 6) Implemente uma função que receba uma string e um caractere, e retorne quantas vezes esse caractere aparece na string. Não faça distinção entre maiúsculas e minúsculas.
- 7) Implemente uma função que copie o conteúdo de uma string para outra, caracter por caracter.
- 8) Implemente uma função que receba uma string e retorne 1 se a string for um palíndromo (lê-se igual de trás para frente), e 0 caso contrário. Não faça distinção entre maiúsculas e minúsculas.
- 9) Implemente uma função que receba uma string e converta todos os seus caracteres para maiúsculos (modificando a própria string).

Exercícios

- 10) Implemente uma função que imprima um vetor de inteiros na vertical, ou seja, cada elemento em uma nova linha no console. Deverá ter o seguinte cabeçalho:

```
void PrintVector(int sz, int vec[sz])
```

- 11) Implemente uma função que faça o swap de dois inteiros. Deverá ter a seguinte assinatura:

```
void Swap(int *a, int *b)
```

- 12) Implemente uma função que recebe um vetor de inteiros e o retorne em ordem crescente.

Deverá utilizar a função Swap (do exercício anterior) e deverá ter o seguinte cabeçalho:

```
void OrdenarCrescente(int sz, int vec[sz])
```

Exercícios

- 13) Implemente uma função que receba um vetor de inteiros e retorne dois vetores: um com os valores pares e outro com os valores ímpares. Deverá ter o seguinte cabeçalho:

```
void SeparaParImpar(int sz, int vec[sz], int vPar[], int vImpar[], int *szPar, int *szImpar)
```

- 14) Implemente uma função que imprime uma matriz na forma padrão (cada linha da matriz em uma nova linha no console). Deverá ter o seguinte cabeçalho:

```
void PrintMatrix(int nRows, int nCols, int matrix[nRows][nCols])
```

- 15) Implemente uma função que recebe duas matrizes de inteiros A e B e, caso seja possível, retornar a multiplicação de $A \times B$ no parâmetro Mult e, no retorno principal, retornar **true** (indicando que a multiplicação foi realizada). Caso não seja possível, a função deverá emitir um aviso e retornar **false** no retorno principal. Deverá ter o seguinte cabeçalho:

```
bool MultMatrix(int linhasA, int colunasA, int A[linhasA][colunasA], int linhasB, int colunasB, int B[linhasB][colunasB], int Mult[linhasA][colunasB])
```

Exercícios

- 16) Faça uma função que implemente um menu para que o usuário escolha qual exercício ele quer rodar. Deverá ter o seguinte cabeçalho:

void EscolherExercicio()

Exemplo de menu:

```
=====
Escolha o exercício que deseja rodar:

1 - Retorna se inteiro eh par.
2 - Le um numero real e retorna a parte fracionaria.
3 - Retorna o fatorial de um ineiro positivo.
4 - Escreve pares entre a e b.
5 - Retorna tamanho de string.
6 - Retorna a qtd de vezes que um caractere aparece na string.
7 - Copia uma string para outra.
8 - Retorna se string é um palíndromo.
9 - Converte todos os caracteres de uma string para maiúsculos.
10 - Imprime vetor.
11 - Faz swap entre dois inteiros.
12 - Ordenação crescente de vetor.
13 - Separa elementos de um vetor em pares e ímpares.
14 - Imprime uma matriz.
15 - Multiplicação de matrizes.
=====
```

A partir da escolha, será utilizado switch case para pedir os dados necessários e chamar a respectiva função.